

Machine Learning in Production

Week 11 · Module 2, Lectures 4–6

CS 5090 · Software Engineering with Generative AI · Week 11 · Module 2, Lectures 4–6

Michigan Technological University · Fall 2026



Lecture 4 · Architecture, Deployment, Operations

Architecture, Deployment, Operations

Making it run, and keep running

- Where the model lives, what it costs to answer, how the pipeline is built, and what it takes to operate the result at three in the morning.
- Project 2 experiments start this week: the five questions on the next slide are also the ones your method section has to answer.



Quality attributes drive the design

Architecture is the set of decisions that are expensive to change later

Attribute	The question
Latency	how long may an answer take? milliseconds or minutes?
Cost	what does one prediction cost, times how many per day?
Privacy	may the data leave the user's device at all?
Availability	what happens when the model service is down?
Updatability	how quickly must a new model reach users?

Quality requirements → architecture and design → implementation. None of the five mentions accuracy: that is a model concern, these are system concerns, and they usually dominate.



Every model sits on a trade-off curve

Prediction error against cost of prediction: there is no best model, only a frontier

- Bigger models are more accurate and slower and dearer per call; the points on the frontier are the only ones worth considering.
- The quality attributes pick the point: a 200 ms budget rules out the most accurate model, a privacy constraint rules out the remote one.
- Architecture is choosing where on this curve to live, and designing so you can move later.

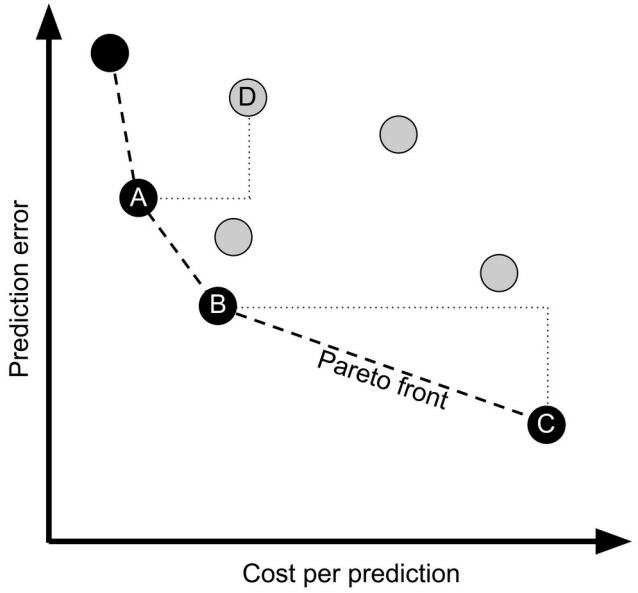


Figure: Kästner, Machine Learning in Production, MIT Press 2025, ch. 9 (CC BY-NC-ND 4.0).

Where does the model live?

Three placements, three sets of trade-offs

Placement	For	Against
Server-side	easy to update, central logs	costs per call, needs network, sees the data
On device	private, offline, free per call	hard to update, limited size
Hybrid	small local model, escalate hard cases: best of both	twice the complexity

The book's running example: an augmented-reality translation app split across cloud, phone, and smart glasses, with bandwidth and battery deciding what runs where.
SE · Your IDE: completion runs local or nearby; the big model runs remote.



The inference path, and the bug that no test catches

Training/serving skew

- Request → features (encode input) → model (predict) → post-process (threshold, guardrail) → response (and log it).
- Training: read CSV → clean → normalise v1 → train. Serving: read JSON → clean → normalise v2 → predict. Result: no error, no alert, just worse, forever.
- The fix is architectural: one shared feature-encoding component, used by both training and serving. Not a test, not a code review, a structure.

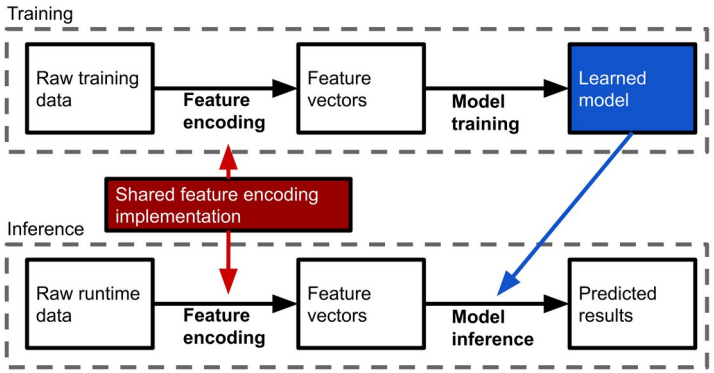


Figure: Kästner 2025, ch. 10. Skew: Sculley et al. 2015; Breck et al. 2017.

Pipeline thinking

Retraining should be a job you run, not a notebook you remember how to use

- Ingest → clean and validate → features (shared code) → train (versioned) → evaluate (a gate) → deploy if it passes.
- Every step reads from and writes to named artefacts; every step can be rerun and tested on its own.
- The evaluate box is a gate, not a report: if the new model is worse on any slice you care about, the pipeline stops on its own.

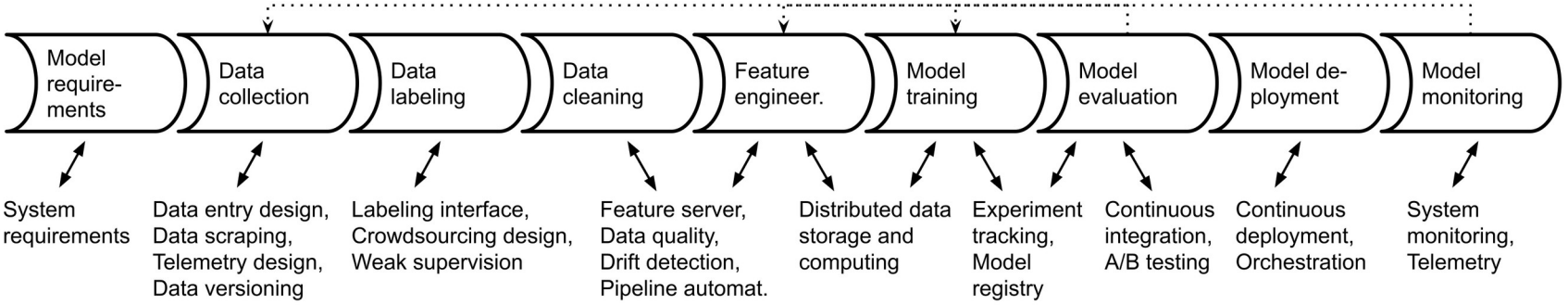


Figure: Kästner 2025, ch. 11.

Operating it

The work that starts after launch and never stops

Practice	What it is
Service-level objectives	a written promise with a number: 99% of answers under 300 ms
Observability	metrics, logs, and traces: could you diagnose it at 3 am?
Infrastructure as code	the environment is in version control, not in someone's head
Incident response	who is paged, what they do, what we learn afterwards



What the operator sees

Latency, throughput, and errors for a model service, on one dashboard

- Three time series the on-call engineer watches: request latency, request rate, error rate. Nothing on this screen is about accuracy.
- Alerts fire on the SLO, not on the model: "p99 latency above 300 ms for five minutes."
- Model-quality monitoring is a separate dashboard, and the topic of Friday's lecture.

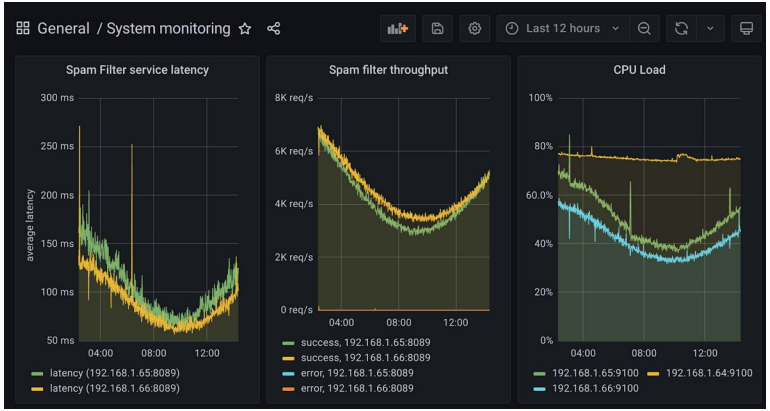


Figure: Kästner 2025, ch. 13.

MLOps in one picture

DevOps, plus the two things ML adds: data and models

- The DevOps loop: plan → code → build → test → release → deploy → operate → monitor, and back to plan.
- What ML adds: data and models are versioned artefacts in the loop alongside code.
- And the monitor step can fire without anyone having changed anything, because the world moved. That is why MLOps is not DevOps with a new name.

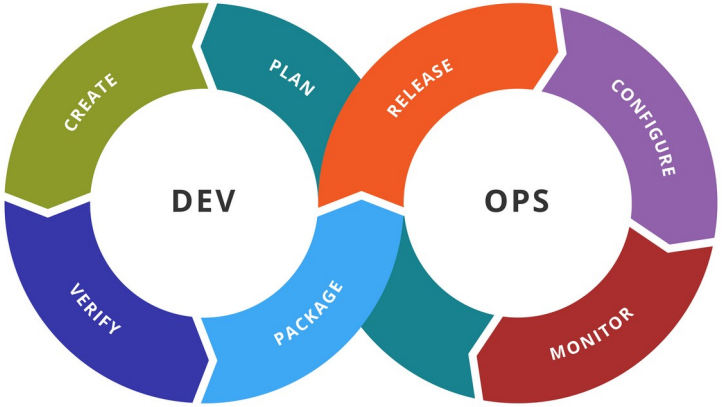
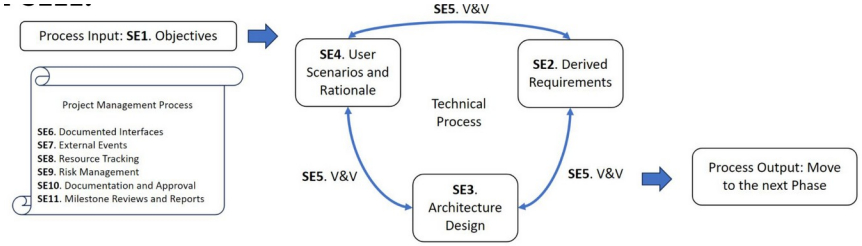


Figure: Kästner 2025, ch. 13. Amershi et al. 2019.

V-model, PR2: architecture is a document with owners

What the interviewed practitioners asked for

- Challenges reported: difficulty moving from a model-centric to a system-wide view; delays in planning and monitoring change.
- **PR2**: a system-level architecture design, with elements, interfaces, responsibilities, alternatives, and expected performances, created and actively maintained as things change, with the owners of both ML and non-ML components participating.
- **PR3**, again: risks such as design changes caused by ML uncertainty are identified and mitigated on purpose.
- In the V: this is SE3 (architecture design), checked by SE5 (V&V) and fed by SE9 (risk management).



Wu, An Exploratory Study of V-Model in Building ML-Enabled Software, CAIN 2024.

Coding view: serving a code model

Applying all of it to the thing you built in Project 1

Tier	Model	Latency
inline completion	small, local	< 200 ms
chat	large, remote	seconds
agent	largest, sandboxed	minutes

Latency budget: inline completion must answer in about 200 ms, so a small model nearby and a large model for explicit requests.
Cost: the KV cache from Module 1 is now a capacity-planning number. Privacy: can this company's code leave the building?
Availability: what does the editor do when the service is down?



Key takeaways

- Latency, cost, privacy, availability, and updatability decide the architecture; accuracy does not appear on the list.
- Where the model lives (server, device, hybrid) is the biggest single decision.
- Training/serving skew is silent and structural; share one feature-encoding path.
- Make retraining a pipeline with an automatic quality gate, not a remembered notebook.
- Operations means SLOs, observability, infrastructure as code, and an incident plan; MLOps is DevOps with data and models in the loop.
- V-model PR2: the architecture is a maintained document with ML and non-ML owners.
- Homework: trace one model from training to serving in any open-source repo. Wednesday: quality assurance at four levels.





Lecture 5 · Quality Assurance at Four Levels

Quality Assurance for ML Systems

Model, data, pipeline, system

- Four levels of quality assurance, because a defect at any one of them reaches the user.
- Second half (⚙️): the same stack applied to code a model wrote.



Four places quality can fail

Testing only the model covers a quarter of the risk

Level	The question
System	does the product do its job, safely?
Pipeline	is the code that builds the model correct?
Data	is the input trustworthy?
Model	are the predictions good?

Most teams test only the narrowest layer. For Project 2, test at least three of the four.



Verification and validation, for a model

Did we build it right, and did we build the right thing?

- Ordinary software: validation checks the system against user needs; verification checks the implementation against the specification.
- ML has no specification. The model is learned from data, so "verification" against a spec is impossible; the test set stands in for the specification, and everything is validation.
- That is why a model's accuracy on a test set is evidence, never proof, and why the other three levels matter.

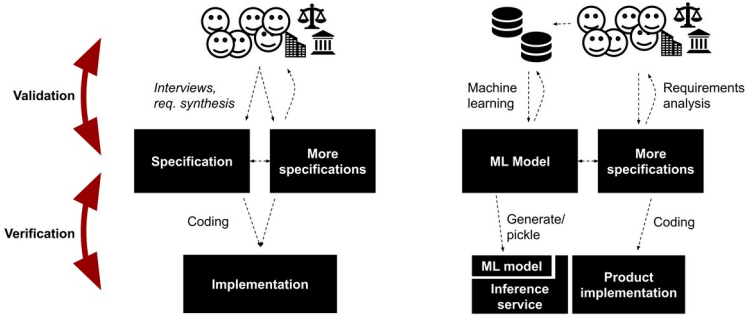


Figure: Kästner, Machine Learning in Production, MIT Press 2025, ch. 15 (CC BY-NC-ND 4.0).

Accuracy hides things

The same model, sliced four ways. The headline number is the least useful one

- 90% overall can be 40% where it matters most. Slice by user group, input length, rare category.
- A random split does not represent tomorrow's inputs: the test data is a sample of the target distribution, which is itself a corner of all possible inputs.
- Curate on purpose: hard cases, edge cases, past failures.

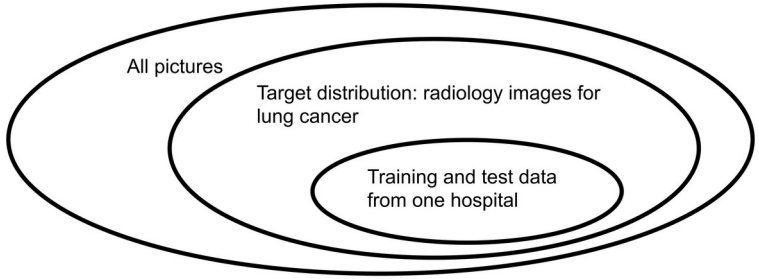


Figure: Kästner 2025, ch. 15. Slicing and behavioural tests: Ribeiro et al., CheckList, ACL 2020.

Behavioural testing

Test capabilities, not just the average, like a unit test suite for a model

Test type	What it checks
Minimum function	the simplest possible case must work
Invariance	irrelevant change in, identical answer out
Directional	this change should move the prediction THIS way
Known failures	every past incident becomes a permanent test case



Data quality and drift

The model file is byte-for-byte identical to launch day, and accuracy still falls

- Schema checks at every boundary · distribution checks · provenance (where did each record come from, may we use it?) · drift monitoring runs forever.
- Data drift: the inputs move. Concept drift: the inputs stay, the right answer moves. Either one degrades a model that never changed.
- SE · Treat the training set as a versioned dependency, like a library.

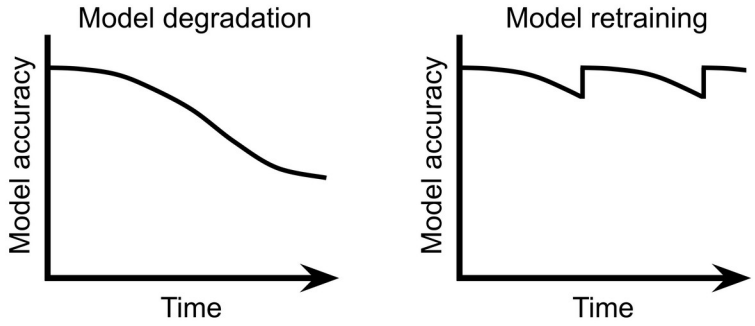


Figure: Kästner 2025, ch. 16. Data cascades: Sambasivan et al., CHI 2021.

Pipeline defects are silent

Code that computes the wrong thing without ever crashing

- A dropped column: training silently loses a feature. A leaked label: 99% accuracy offline, useless in production. A wrong join: rows misaligned, everything shifts by one.
- None of these throw an exception. Ordinary bug: crashes, you fix it. Pipeline bug: no crash, worse model.
- SE · Unit-test the pipeline like any other code: drivers and stubs around each stage, assertions on the data that flows between them.

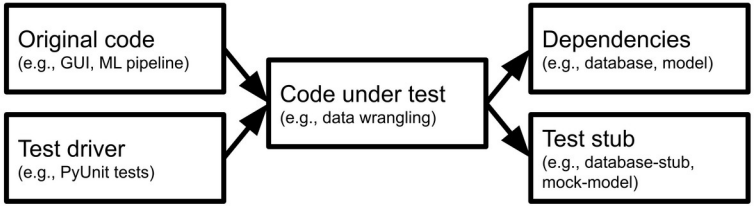


Figure: Kästner 2025, ch. 17. Breck et al., The ML Test Score, 2017.



System-level assurance

The parts can each be fine and the whole still fail

- Test the safeguards, not only the model: does the guardrail actually fire?
- Test the failure paths: model timeout, model unavailable, garbage response.
- Test the operations: does the alert reach a human? does rollback work?
- Run the whole product against realistic scenarios, with the ML and non-ML components wired together as they will be in production.

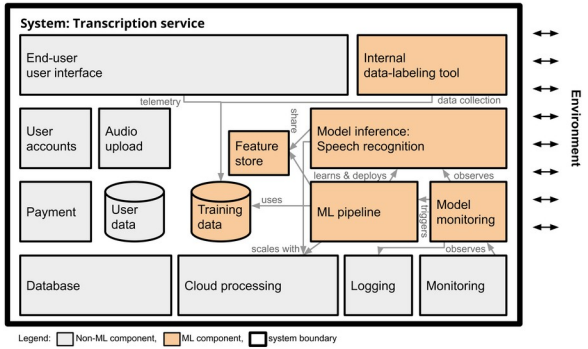


Figure: Kästner 2025, ch. 18.

V-model, PR5 and PR6: data is a component, V&V at every level

What the interviewed practitioners asked for

- Challenges reported: model teams assumed no responsibility for testing the product; product teams had no planned system-level testing; data quality depended on domain experts nobody had scheduled.
- **PR5**: data is a separate component with its own requirements, design, and validation (data validation and data monitoring), beside the ML and non-ML components.
- **PR6**: verification and validation at system, subsystem, and component level (ML, non-ML, data, infra), with named owners.
- The classic V from software engineering already has this shape: each design level on the left is checked by a test level on the right.

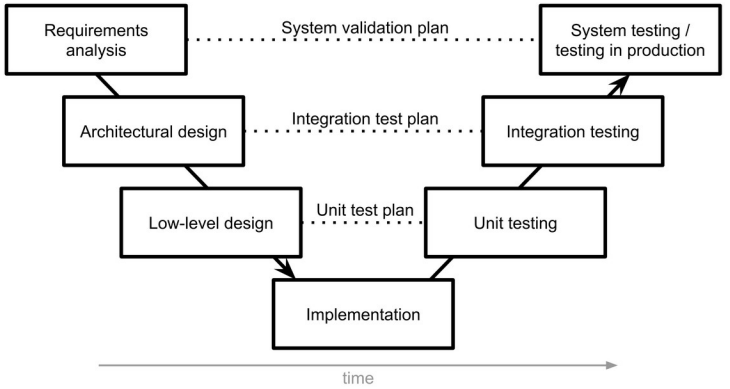


Figure: Kästner 2025, ch. 14. Propositions: Wu, CAIN 2024.

⚙️ How generated code fails

The distribution is different from human-written code

Failure	What it looks like
Plausible but wrong	reads beautifully, subtly incorrect: the dominant mode
Invented APIs	a method that should exist, and does not
Silently outdated	the idiom from three library versions ago
Missing error handling	the happy path, immaculately written
Over-engineering	abstraction layers nobody asked for



⚙️ The assurance stack

Cheap and certain at the bottom; expensive and judgemental at the top

- Top to bottom: human review (judgement, design, intent) → tests and mutation (behaviour you specified) → static analysis and security scanning (whole classes of defect) → types and compilation (instant, free, certain).
- Push every check as far down as it will go. Human attention is the scarce resource.
- The figure: static-analysis warnings shown inline at code review, the way Google's Tricorder does it. The reviewer never spends attention on what a tool already found.



Figure: Kästner 2025, ch. 14 (Tricorder, Sadowski et al. 2015).

Key takeaways

- Quality fails at four levels: model, data, pipeline, and system. Test at least three.
- A model cannot be verified, only validated: one accuracy number hides everything; slice it and curate hard test sets.
- Behavioural tests give you a test suite where no specification exists.
- Data and concept drift degrade a model that never changed; monitor forever.
- Pipeline defects are silent; they need review and assertions, not exceptions.
- Test the safeguards, the failure paths, and the operations, not only the model; V-model PR5 and PR6: data is a component, every level has an owner.
- ⚙️ Generated code fails plausibly; push every check down the assurance stack.
- Friday: testing in production, versioning. Last day of talks.





Lecture 6 · Testing in Production, Versioning

Testing in Production, Versioning, MLOps

Learning from real users

- You cannot learn everything offline.
- Measuring safely on live users, and being able to explain, months later, exactly what produced a given answer.
- Your Project 2 experiments started this week: the release procedure at the end of this lecture is a template for them.



Why offline is not enough

Every piece of software is tested in production; the question is whether on purpose

- Your test set is a guess about the future. Real inputs are stranger than any test set you would think to build.
- Real users respond to the system, changing the data it sees.
- Offline metrics measure the model; only production measures the product.
- The question is not whether to test in production. It is how to do it safely. Netflix, Google, and every serious ML team run experiments continuously.



Figure: Kästner 2025, ch. 19. Kohavi, Tang & Xu, Trustworthy Online Controlled Experiments, 2020.

Four safe ways to ship

Each limits blast radius differently

Method	What it does
Shadow	run the new model in parallel, serve the old: zero user risk
Canary	1% of traffic, watch the metrics, expand or roll back
A/B test	split traffic deliberately to measure a difference
Feature flag	turn it off in seconds without a deployment

The canary is the miner's bird: a small, cheap, early warning before the whole mine is at risk.



A/B testing, briefly

The experiment behind most Project 2 claims

- Randomly assign users (or requests) to A and B; decide the metric and the sample size before you start.
- Statistical significance: with enough samples, small real differences become visible; with few samples, large noise looks like a difference.
- Do not peek and stop early: stopping when the result first looks good inflates false positives.
- Report the difference with its uncertainty, not a single number. The same rule as pass@k with n and k reported (Week 8).

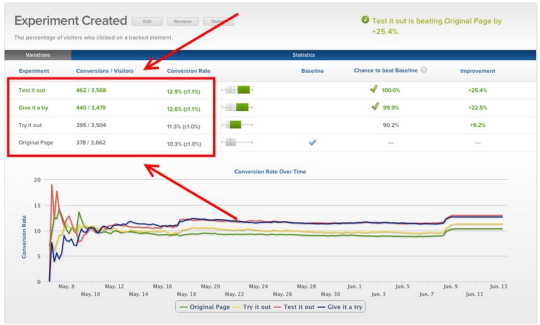


Figure: Kästner 2025, ch. 19.

Telemetry has to be designed

You cannot measure what you did not instrument, and you cannot log everything

- Decide the question BEFORE you decide the log line.
- Sample: full logging is expensive and often illegal.
- Look for a natural label: did the user accept, undo, retry, or leave?
- Privacy is a design constraint, not an afterthought: users can see and switch off what you collect.
- SE · The best telemetry costs the user nothing and tells you everything.

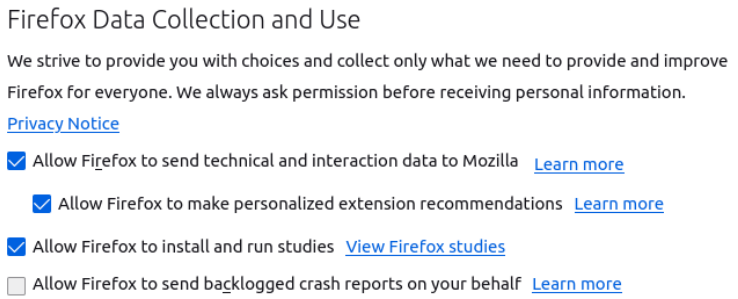


Figure: Kästner 2025, ch. 19.

Monitoring the model, not just the service

Monday's dashboard watched latency; this one watches whether the answers are still good

- Plot a model-quality signal over time: accepted suggestions, undo rate, a proxy label, or a sampled human check.
- Alert on a drop, the same way the SLO alerts on latency. This is the drift monitoring Wednesday asked for.
- Separate dashboards for the service (is it up?) and the model (is it right?); different people are paged.

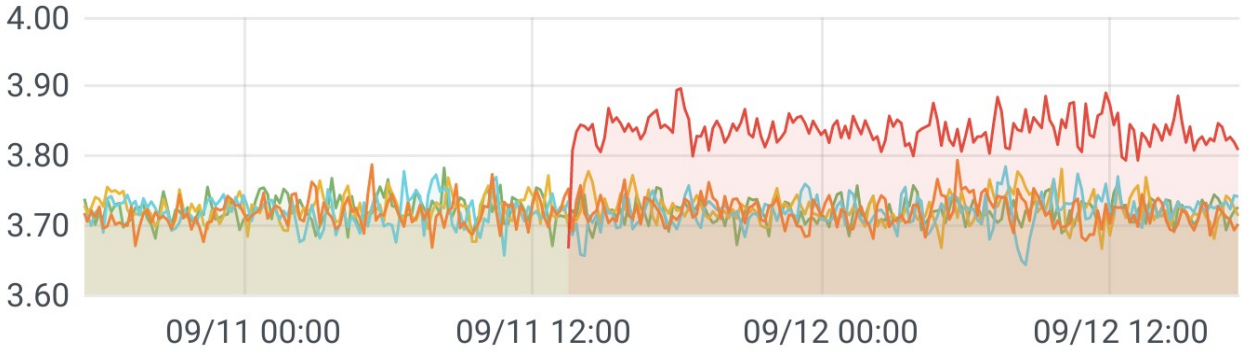


Figure: Kästner 2025, ch. 19.

Feedback loops

The system changes the world, then learns from the world it changed

- Model recommends → users see and click → clicks become training data → model recommends more of that → …
- A small bias amplifies with every cycle. It is a systems problem, not a model problem.
- Break it with randomised exploration, and by holding out data the model did not influence.

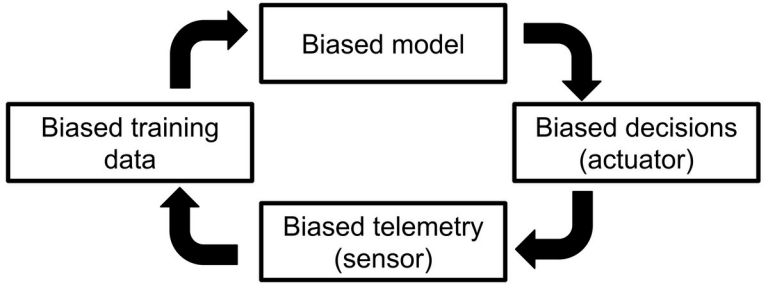


Figure: Kästner 2025, ch. 26. Sculley et al. 2015.

Four things to version

Reproducibility means being able to rebuild any answer you ever gave

Artefact	How
Code	ordinary version control: the easy one
Data	the training set is a dependency; pin it and hash it
Model	the artefact plus the exact config that produced it
Environment	library versions, hardware, random seeds

The application depends on a model version; the model version depends on a pipeline version and a data version. Every arrow carries a number.



Provenance

"The Apple Card is such a sexist program": can the company answer?

- Walk the chain right to left: offer ← model version ← training run and config ← data snapshot ← where each record came from and under what licence.
- Break any link and the question "why did this customer get this answer?" is unanswerable.

The five-minute fix: log the model version with every prediction.

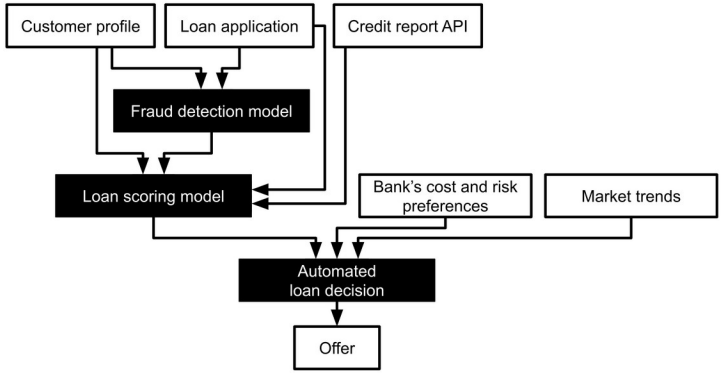


Figure: Kästner 2025, ch. 24. The 2019 Apple Card case: the issuer could not explain why spouses with shared finances got different limits.

Coding view: shipping a model update

The whole lecture as one release procedure, and a Project 2 experiment template

- Shadow: compare the new model against the current one on real requests, offline.
- Canary 1%: watch acceptance.
- Expand: if survival of accepted code holds.
- Flag off: if not.
- The metric watched is survival, not acceptance: a model that gets more suggestions accepted and more of them reverted is a regression.
- The same shape for your experiment: baseline, treatment, the metric decided in advance, a stopping rule.



Key takeaways

- Offline metrics measure the model; only production measures the product.
- Ship safely with shadow, canary, A/B, and feature flags, never all at once; decide the metric and sample size before an A/B test, and do not peek.
- Design telemetry from the question down, and look for natural labels; monitor the model, not only the service.
- Feedback loops amplify bias; break them with exploration and untainted data.
- Version code, data, model, and environment, and log the version with every prediction.
- Homework: find one public post-mortem of an ML failure in production; bring the link. Monday: process, teams, technical debt.

